

Углубленное машинное обучение и искусственный интеллект в Python

Патласов Дмитрий Александрович, ПГНИУ
dmitriypatlasov@gmail.com

№ п/п	Наименование разделов, дисциплин (модулей)	Общая трудоемк ость, ч	Всего, ауд. ч	Аудиторные занятия, ч		Дистанционные занятия, ч		Самост. работа, ч	Форма контроля
				лекции	практ.	лекции	практ.		
1	2	3	4	5	6	7	8	9	10
1.	Основные архитектуры нейронных сетей	16	10	–	–	4	6	6	Практич. задание
1.1.	Полносвязные нейронные сети, подбор гиперпараметров	5	3	–	–	1	2	2	
1.2.	Рекуррентные нейронные сети	6	4	–	–	2	2	2	
1.3.	Сверточные нейронные сети	5	3	–	–	1	2	2	
2.	Специальные архитектуры и механизмы внимания	11	7	–	–	3	4	4	Практич. задание
2.1.	Трансформер архитектура	6	4	–	–	2	2	2	
2.2.	Генеративные нейронные сети	5	3	–	–	1	2	2	
3.	Предобученные модели, интерпретация результатов и автономные AI-системы	22	14	–	–	6	8	8	Практич. задание
3.1.	Предобученные сети для решения задач NLP	6	4	–	–	2	2	2	
3.2.	Предобученные сети для решения задач Time Series	5	3	–	–	1	2	2	
3.3.	Интерпретация результатов обучения нейронных сетей, explainable AI	6	4	–	–	2	2	2	
3.4.	AI-агенты	5	3	–	–	1	2	2	
4.	Разработка и реализация проекта	7	–	–	–	–	–	7	Практич. задание
5.	Итоговая аттестация	4	4	–	–	–	4	–	Зачет

Как пользоваться Python локально WSL2 (Windows Subsystem for Linux 2)

WSL2 (Windows Subsystem for Linux 2) — это архитектурное расширение операционной системы Windows NT, разработанное корпорацией Microsoft, которое обеспечивает бинарную совместимость на уровне ядра между Windows и Linux-окружением.

Архитектурные особенности и принцип работы

- Уровень абстракции: В отличие от первой версии WSL, выполнявшей трансляцию системных вызовов (syscall translation), WSL2 использует аппаратную виртуализацию.
- Микрогипервизор: Запуск сред Linux осуществляется внутри облегченной виртуальной машины (Utility VM) под управлением модифицированной версии гипервизора Hyper-V.
- Полноценное ядро: Внутри вирт-пространства функционирует кастомное, высокооптимизированное ядро Linux, компилируемое из стабильных исходных кодов kernel.org.
- Динамическое распределение: Выделение оперативной памяти (RAM) и процессорного времени (CPU) происходит динамически по требованию, минимизируя накладные расходы хост-системы.

Ключевые преимущества для Data Science и TensorFlow

- Нативная поддержка файловых систем: Использование протокола 9P для межсистемного взаимодействия обеспечивает высокую скорость операций ввода-вывода (I/O) в файловой системе ext4.
- Прямой доступ к аппаратным ресурсам (GPU Passthrough): Поддержка интерфейсов API DirectX 12 и NVIDIA CUDA позволяет контейнеризированным и локальным Linux-приложениям задействовать мощности графических процессоров хоста с минимальной потерей производительности.
- Среда выполнения Docker: Позволяет разворачивать легковесные контейнеры Linux без необходимости эмуляции полноценной ОС.

WSL

<https://learn.microsoft.com/ru-ru/windows/wsl/install>

```
dmitry@DESKTOP-3V2H5DK:/mnt/c/Users/dmitry$ neofetch
./+oossssoo+/-./
`:+ssssssssssssssssss+:`
-+ssssssssssssssssssyyssst-
.ossssssssssssssssssdMMMNyssso.
/sssssssssshdmmNNmmyNMMMMhssssss/
+ssssssssshmydMMMMMMNddddyssssssst+
/sssssssshNMMMyhhyyyhmNMMMNhssssssss/
.ssssssssdMMMNhssssssssshNMMMdssssssss.
+sssshhhyNMMNysssssssssssyNMMMyssssssst+
ossyNMMMNyMMhssssssssssshmmnhssssssso
ossyNMMMNyMMhssssssssssshmmnhssssssso
+sssshhhyNMMNyssssssssssyNMMMyssssssst+
.ssssssssdMMMNhssssssssshNMMMdssssssss.
/sssssssshNMMMyhhyyyhdNMMMNhssssssss/
+ssssssssdmydMMMMMMNddddyssssssst+
/sssssssssshdmmNNNmyNMMMMhssssss/
.ossssssssssssssssssdMMMNyssso.
-+ssssssssssssssssssyyyssst-
`:+ssssssssssssssssss+:`
./+oossssoo+/-./
```

```
dmitry@DESKTOP-3V2H5DK
-----
OS: Ubuntu 24.04.1 LTS on Windows 10 x86_64
Kernel: 6.6.87.2-microsoft-standard-WSL2
Uptime: 16 hours, 56 mins
Packages: 662 (dpkg)
Shell: bash 5.2.21
Theme: Adwaita [GTK3]
Icons: Adwaita [GTK3]
Terminal: Windows Terminal
CPU: Intel Ultra 7 265KF (20) @ 3.878GHz
GPU: ede8:00:00.0 Microsoft Corporation Device 008a
Memory: 1365MiB / 15803MiB
```



Инструкция по развертыванию WSL 2 с дистрибутивом Ubuntu LTS на Windows 11 через интерфейс командной строки.

Шаг 1. Запуск терминала с правами администратора

1. Нажмите клавиши `Win + X`.
2. Выберите пункт **Терминал (Администратор)** или PowerShell (Администратор). [Microsoft Learn +1](#)

Шаг 2. Проверка доступных дистрибутивов (Опционально)

Чтобы увидеть точные системные имена доступных версий Ubuntu LTS, выполните команду: [Pureinfotech +1](#)

powershell

```
wsl --list --online
```

Use code with caution.

В выведенном списке найдите необходимую LTS-версию (например, Ubuntu-24.04 или актуальную Ubuntu по умолчанию). [blog.gordonbuchan.com +1](#)

Шаг 4. Перезагрузка хост-системы

После завершения скачивания ядра Linux и компонентов виртуализации терминал потребует перезагрузить компьютер. Выполните перезагрузку через меню «Пуск» или напрямую из терминала: [Pureinfotech +1](#)

powershell

```
Shutdown /r /t 0
```

Use code with caution.

Шаг 5. Инициализация Ubuntu LTS

1. После перезагрузки ОС Windows автоматически откроется консольное окно Ubuntu для завершения установки.
2. Система предложит создать уникальную учетную запись Linux:
 - **Enter new UNIX username:** введите имя пользователя строчными буквами (например, `dmitry`).
 - **New password:** введите безопасный пароль (символы при вводе отображаться не будут) и нажмите `Enter`. [OneUptime +1](#)

Шаг 3. Установка WSL и Ubuntu LTS

Для установки **актуальной LTS-версии по умолчанию** выполните базовую команду: [OneUptime +1](#)

powershell

```
wsl --install
```

Use code with caution.

Примечание: По умолчанию Microsoft разворачивает именно релизы Ubuntu. Если вам требуется **конкретная LTS-версия** (например, 24.04), укажите её имя через флаг `-d`: [Pureinfotech +2](#)

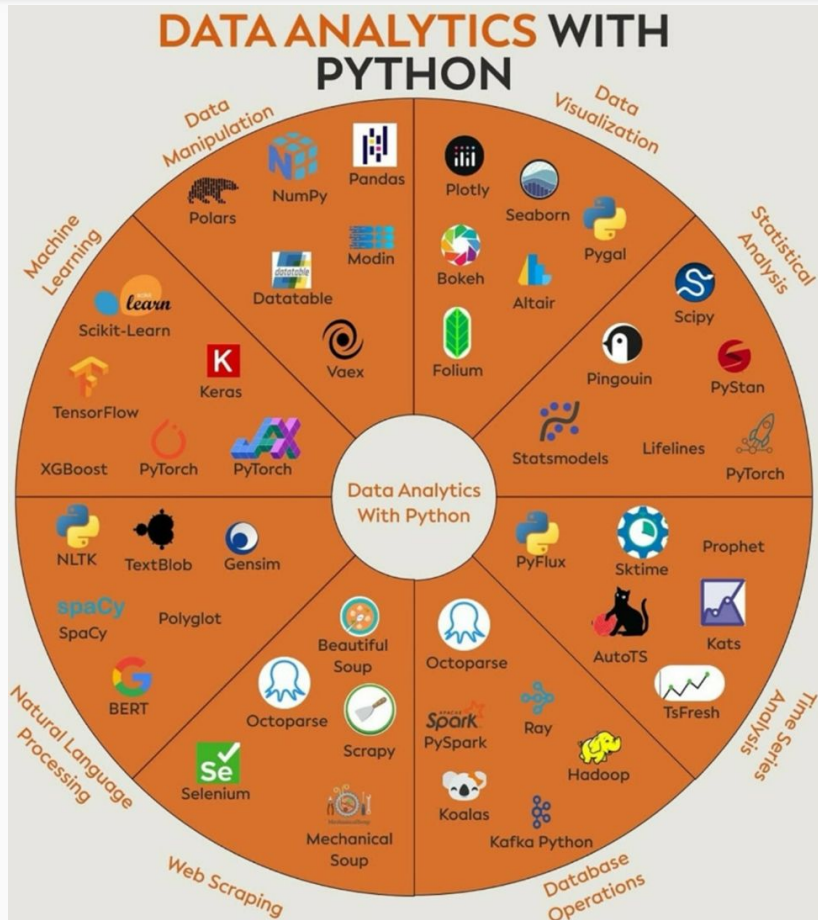
powershell

```
wsl --install -d Ubuntu-24.04
```

Use code with caution.

```
PS C:\Users\dmitry> wsl --list --verbose
  NAME                STATE      VERSION
* Ubuntu-24.04        Running    2
PS C:\Users\dmitry> |
```

Фреймворки для построения нейронных сетей с нуля (почти)



PyTorch

TensorFlow

K Keras

Tensorflow

<https://www.tensorflow.org/install?hl=ru>



```
# Requires the latest pip
$ pip install --upgrade pip

# Current stable release for CPU
$ pip install tensorflow

# Current stable release for GPU (Linux / WSL2)
$ pip install tensorflow[and-cuda]

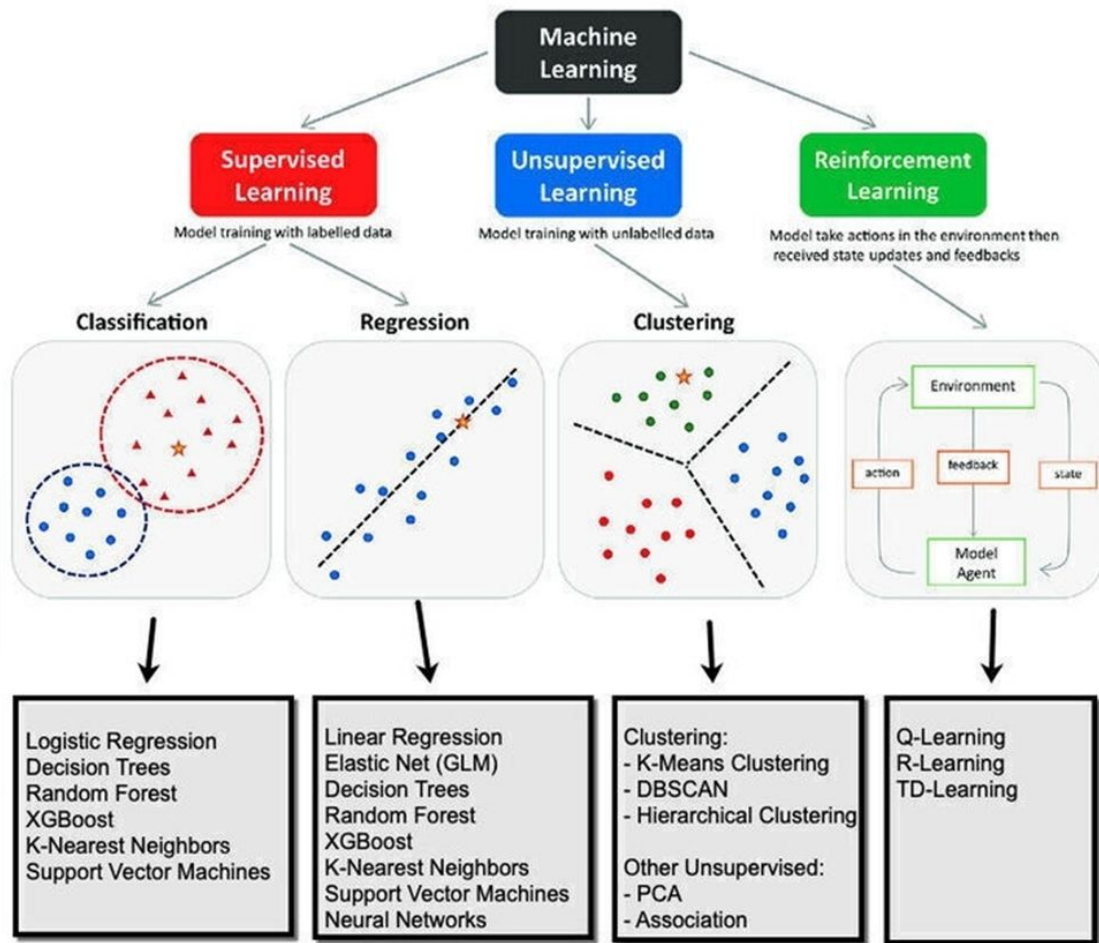
# Or try the preview build (unstable)
$ pip install tf-nightly
```

Torch <https://pytorch.org/>

NOTE: Latest Stable PyTorch requires Python 3.10 or later.

PyTorch Build	Stable (2.12.0)		Preview (Nightly)		
Your OS	Linux	Mac	Windows		
Package	Pip	LibTorch	Source		
Language	Python		C++ / Java		
Compute Platform	CUDA 12.6	CUDA 13.0	CUDA 13.2	ROCm 7.2	CPU
Run this Command:	<pre>pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu132</pre>				

3 Types of Machine Learning



*Тестовая и обучающая выборки

- Входные данные и метки
- Тестовая и обучающая части набора данных

```
with open('cifar-100-python/train', 'rb') as f:  
    data_train = pickle.load(f, encoding='latin1')  
with open('cifar-100-python/test', 'rb') as f:  
    data_test = pickle.load(f, encoding='latin1')
```

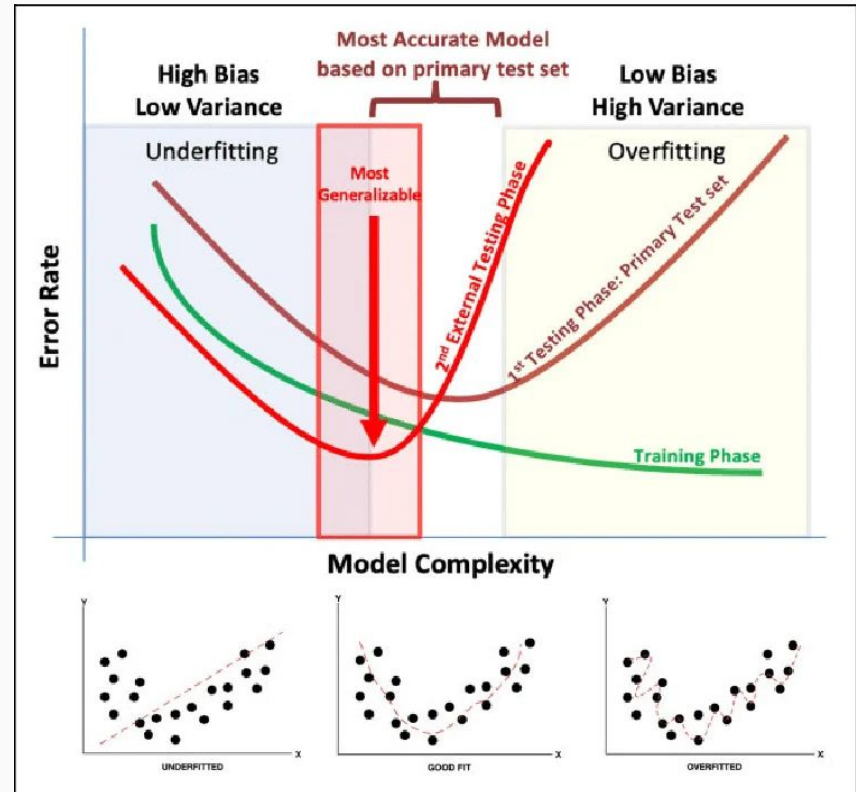


Dataset Viewer			Auto-
Split (3)			
train · 980k rows			▼
Split (3)			
train (980k rows)			
validation (10k rows)			
test (10k rows)			
36	36		6
d214632e-eb17-	*aW1wb3J0IGNhZH1ZXJ5IGFzIGNxCgpmX2JvZH1fbGVuZ3RoI		

Bias-variance tradeoff

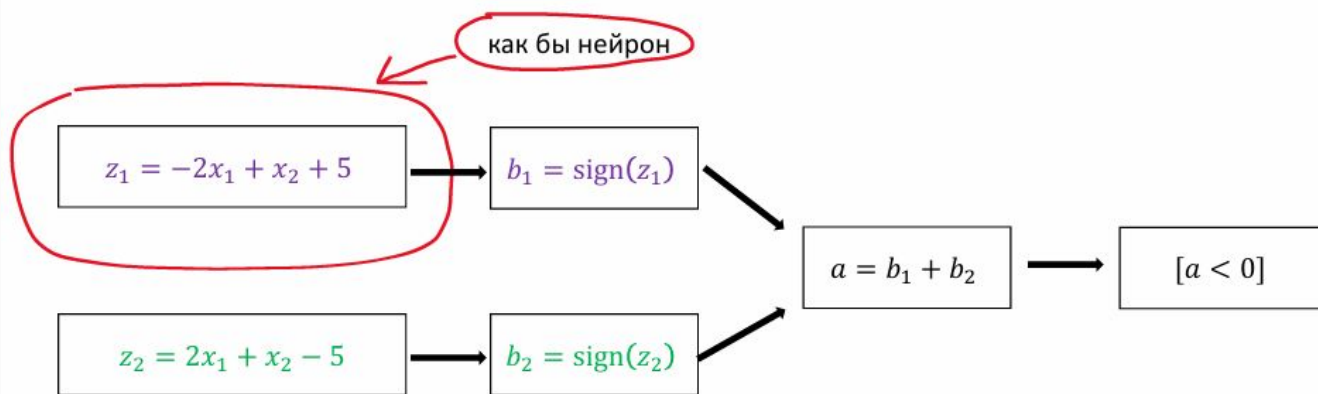
Смещение и разброс (Bias and variance)

- Смещение: ожидаемая разница между прогнозами модели и истиной
- Разброс: Как сильно отличается ответ модели в обучающих наборах данных
- Возможные сценарии модели
 - Большое смещение: модель делает неправильный прогноз на обучающих примерах
 - Большой разброс: Модель не обобщает на новых наборах данных
 - Маленькое смещение: Модель делает правильный прогноз на обучающих данных
 - Маленький разброс: Модель обобщает на новых наборах данных



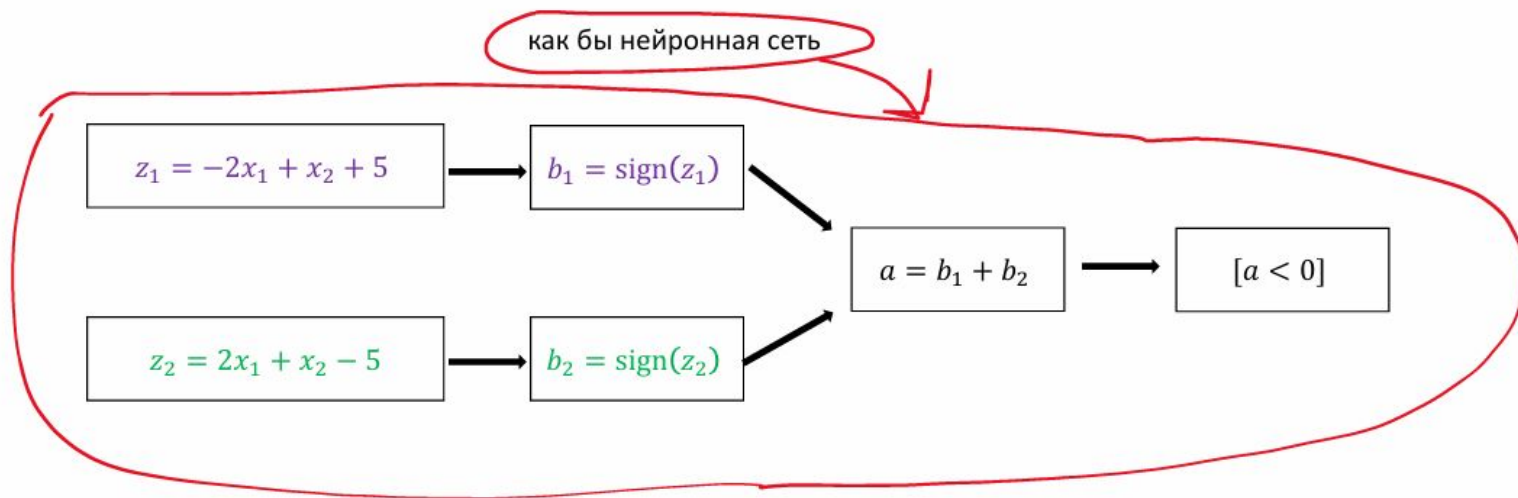
Зачем нужна нейронная сеть

Нелинейные закономерности



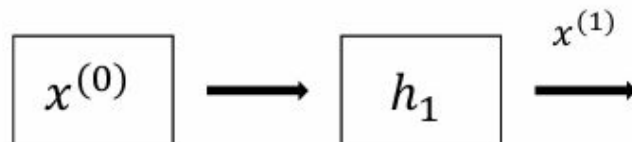
Зачем нужна нейронная сеть

Нелинейные закономерности

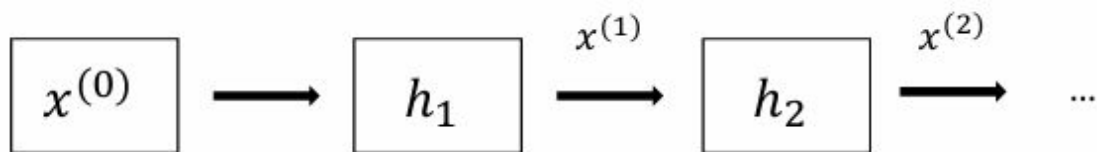


Граф вычислений (или нейронная сеть)

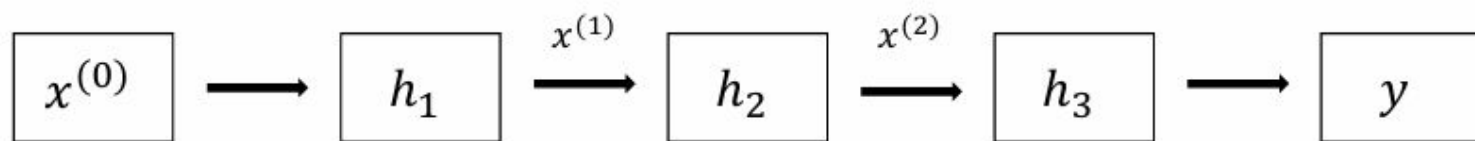
- $x^{(0)}$ — признаки объекта
- $h_1(x)$ — преобразование («слой»)
- $x^{(1)}$ — результат



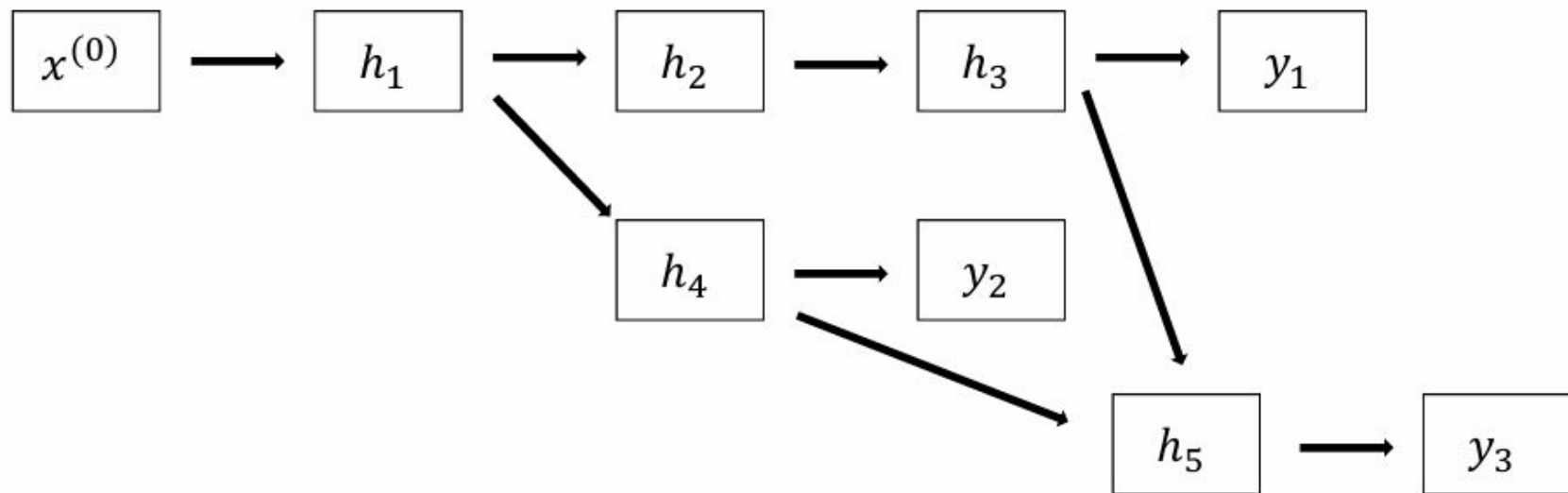
Граф вычислений (или нейронная сеть)



Граф вычислений (или нейронная сеть)



Граф вычислений (или нейронная сеть)



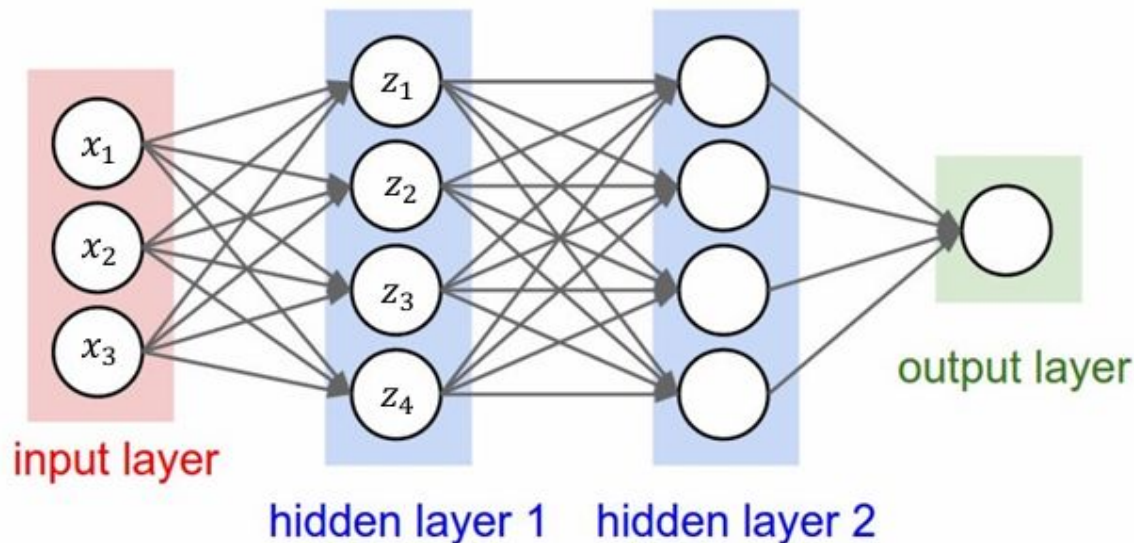
Полносвязный слой (fully connected, FC)

- На входе n чисел, на выходе m чисел
- $x_1, \dots, x_n$ — ВХОДЫ
- $z_1, \dots, z_m$ — ВЫХОДЫ
- Каждый выход — линейная модель над входами

$$z_j = \sum_{i=1}^n w_{ji} x_i + b_j$$

Полносвязный слой (fully connected, FC)

$$z_j = \sum_{i=1}^n w_{ji} x_i + b_j$$



Полносвязный слой (fully connected, FC)

$$z_j = \sum_{i=1}^n w_{ji} x_i + b_j$$

- t линейных моделей, в каждой $(n + 1)$ параметров
- Всего примерно tn параметров в полносвязном слое

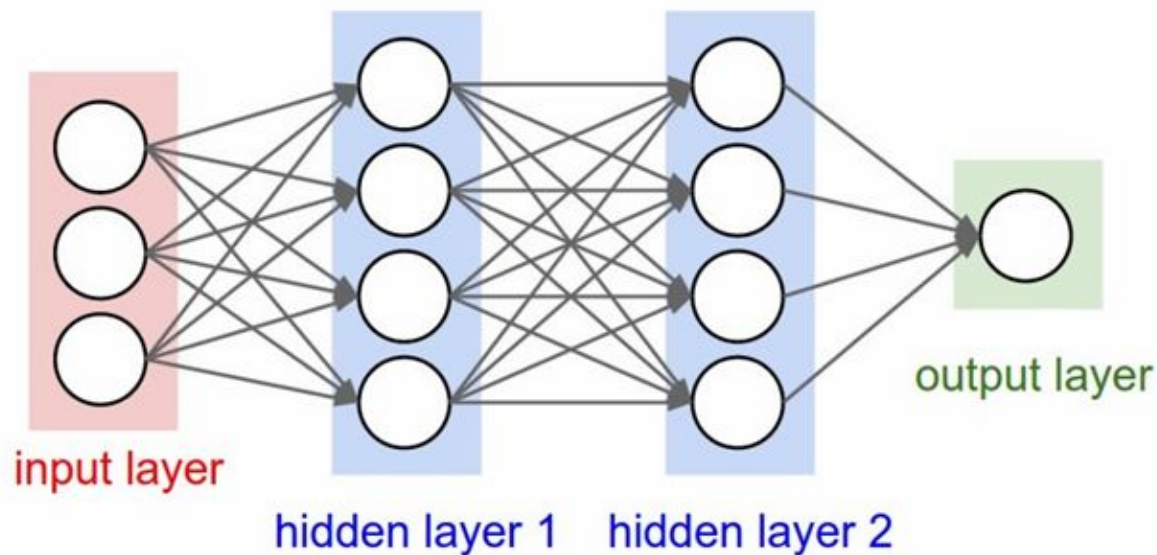
Полносвязный слой (fully connected, FC)

$$z_j = \sum_{i=1}^n w_{ji}x_i + b_j$$

- m линейных моделей, в каждой $(n + 1)$ параметров
- Всего примерно mn параметров в полносвязном слое
- Это очень много: если у нас 1.000.000 входных признаков и 1000 выходов, то это 1.000.000.000 параметров
- Надо много данных для обучения

Нелинейность

- Рассмотрим два полносвязных слоя



Нелинейность

- Рассмотрим два полносвязных слоя

$$\begin{aligned} s_k &= \sum_{j=1}^m v_{kj} z_j + c_k = \sum_{j=1}^m v_{kj} \sum_{i=1}^n w_{ji} x_i + \sum_{j=1}^m v_{kj} b_j + c_k = \\ &= \sum_{j=1}^m \left(\sum_{i=1}^n v_{kj} w_{ji} x_i + v_{kj} b_j + \frac{1}{m} c_k \right) \end{aligned}$$

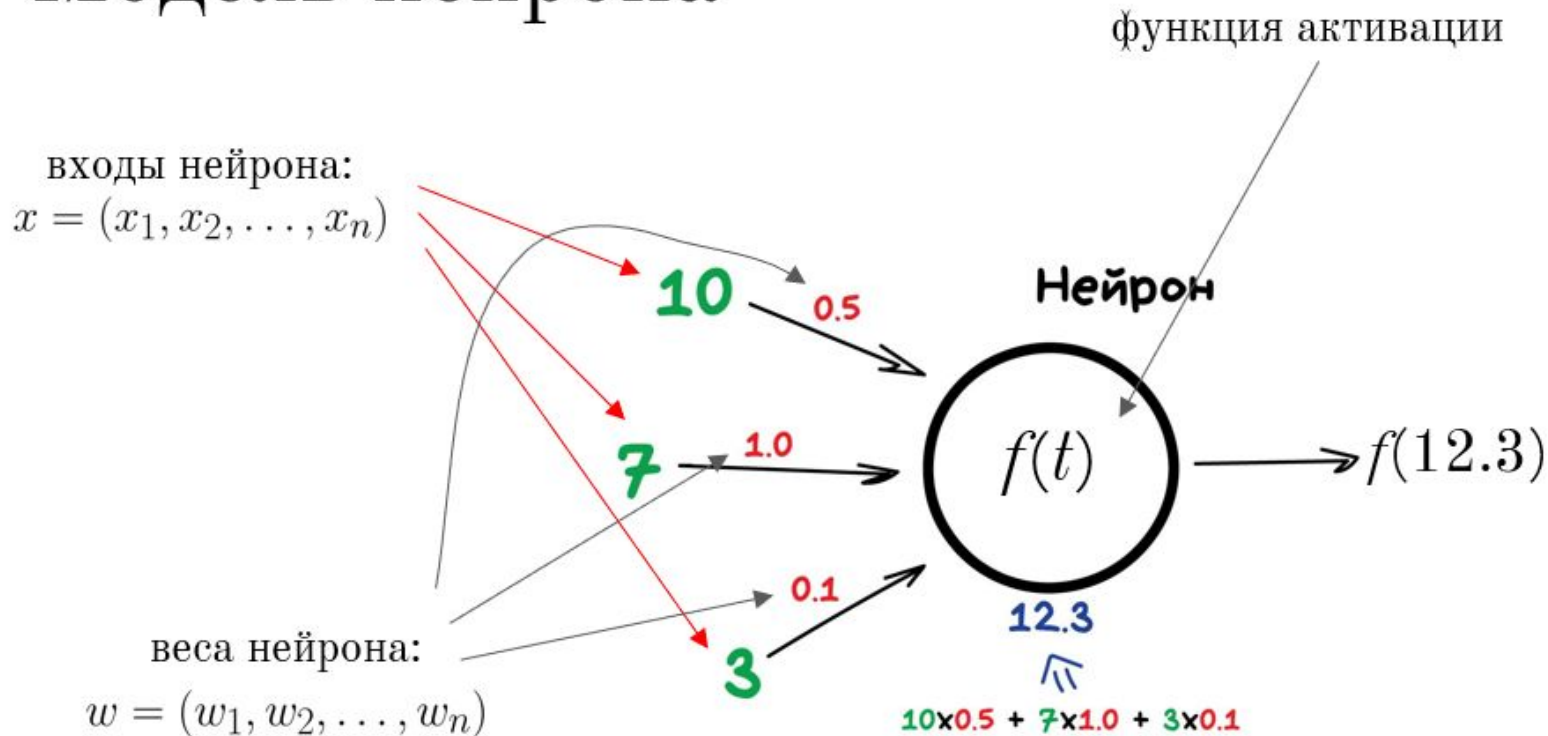
- То есть это ничем не лучше одного полносвязного слоя

Нелинейность

- Нужно добавлять нелинейную функцию после полносвязного слоя

$$z_j = f \left(\sum_{i=1}^n w_{ji} x_i + b_j \right)$$

Модель нейрона



скалярное произведение векторов x, w :

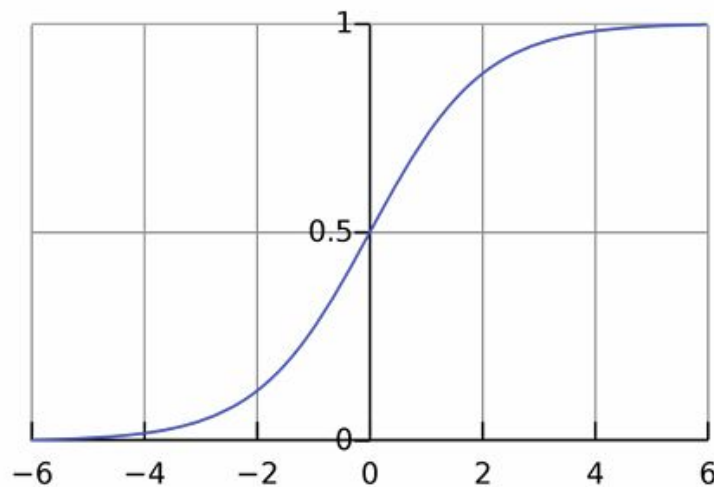
$$w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n = \langle w, x \rangle$$

Нелинейность

$$z_j = f\left(\sum_{i=1}^n w_{ji}x_i + b_j\right)$$

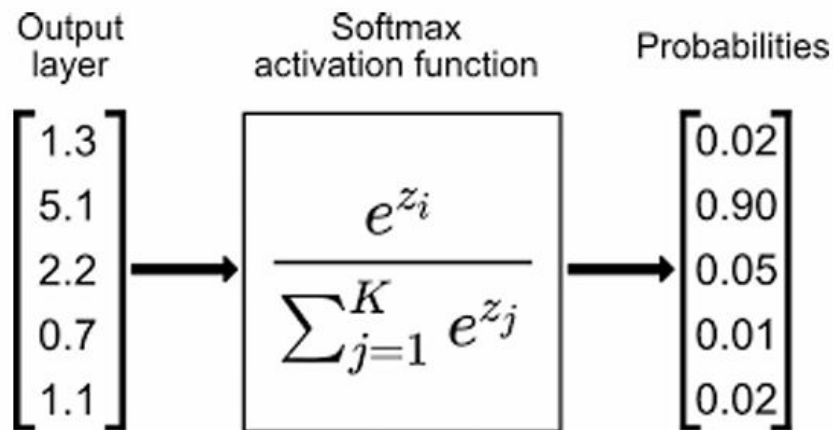
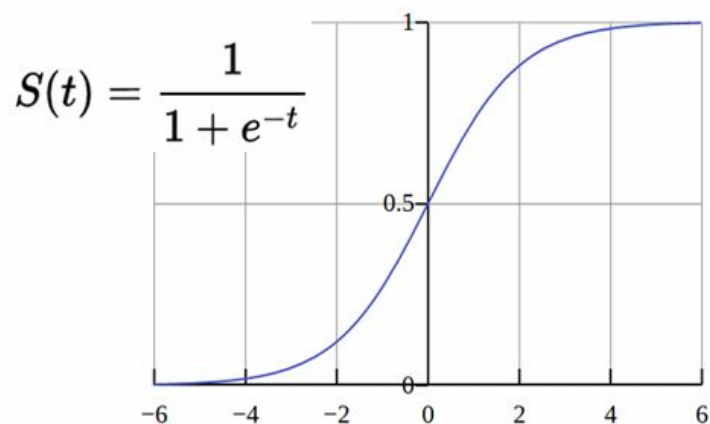
Вариант 1: $f(x) = \frac{1}{1+\exp(-x)}$

(сигмоида)



*Активационные функции

- Активационные функции применяются ко всем входным значениям в каждом нейроне
 - Сигмоидальная функция является распространенной активационной функцией
 - Softmax – это вариант сигмоиды для нескольких классов

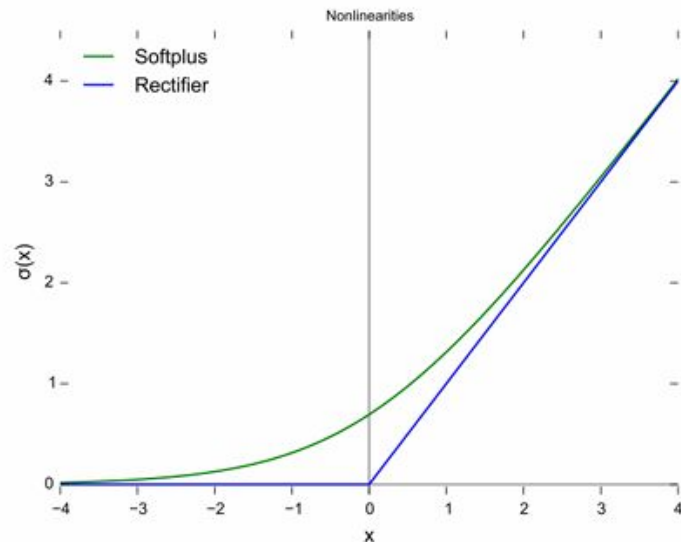


Нелинейность

$$z_j = f\left(\sum_{i=1}^n w_{ji}x_i + b_j\right)$$

Вариант 2: $f(x) = \max(0, x)$

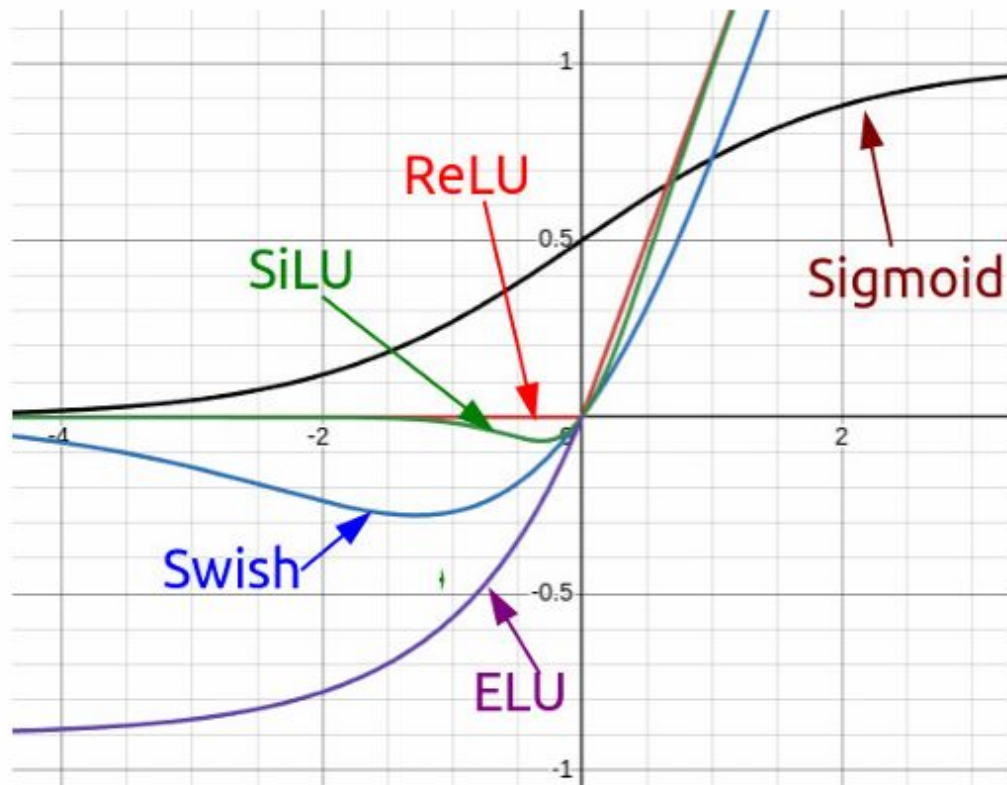
(ReLU, REctified Linear Unit)



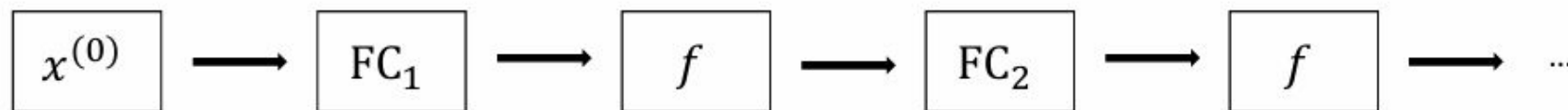
Нелинейность

Rectified linear unit (ReLU) ^[9]		$\begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$ $= \max\{0, x\} = x \mathbf{1}_{x>0}$
Gaussian Error Linear Unit (GELU) ^[4]		$\frac{1}{2}x \left(1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right) \right)$ $= x\Phi(x)$
Softplus ^[10]		$\ln(1 + e^x)$
Exponential linear unit (ELU) ^[11]		$\begin{cases} \alpha(e^x - 1) & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$ with parameter α
Scaled exponential linear unit (SELU) ^[12]		$\lambda \begin{cases} \alpha(e^x - 1) & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$ with parameters $\lambda = 1.0507$ and $\alpha = 1.67326$
Leaky rectified linear unit (Leaky ReLU) ^[13]		$\begin{cases} 0.01x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$
Parameteric rectified linear unit (PReLU) ^[14]		$\begin{cases} \alpha x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$ with parameter α
Sigmoid linear unit (SiLU, ^[4] Sigmoid shrinkage, ^[15] SiL, ^[16] or Swish-1 ^[17])		$\frac{x}{1 + e^{-x}}$

Нелинейность



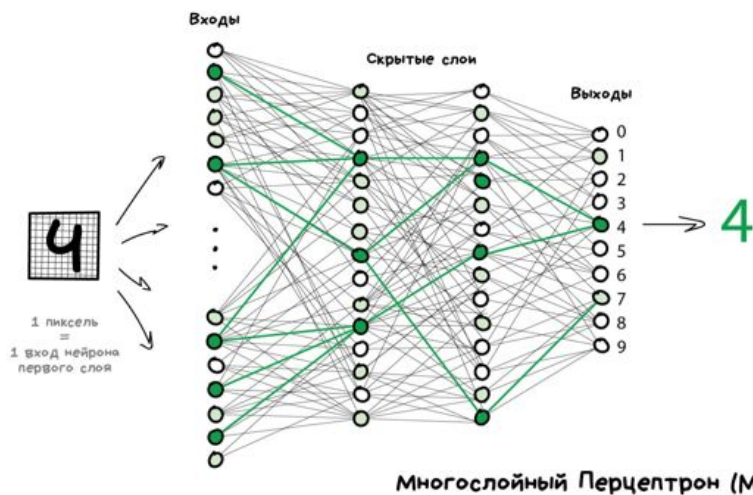
Типичная полносвязная сеть



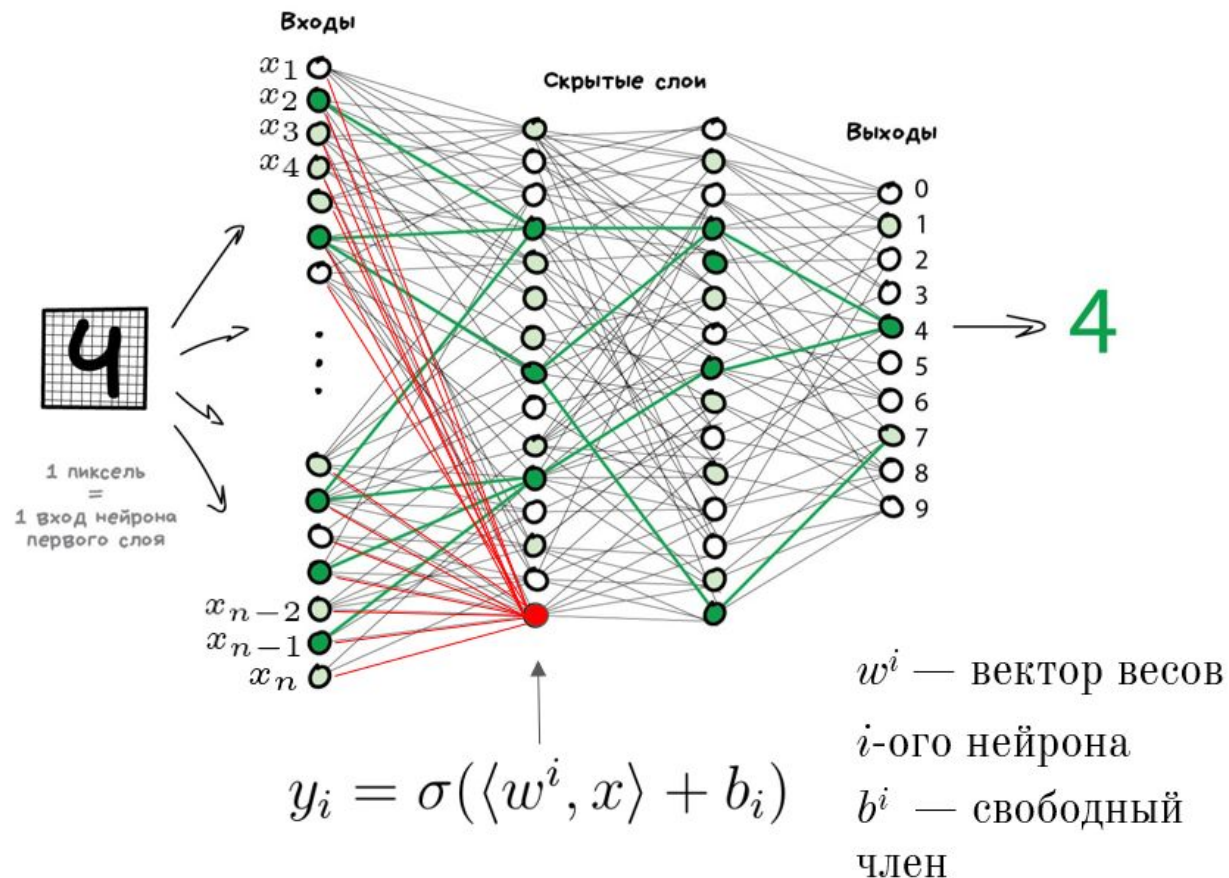
- На входе признаки
- В последнем слое выходов столько, сколько целевых переменных мы предсказываем

Многослойный перцептрон

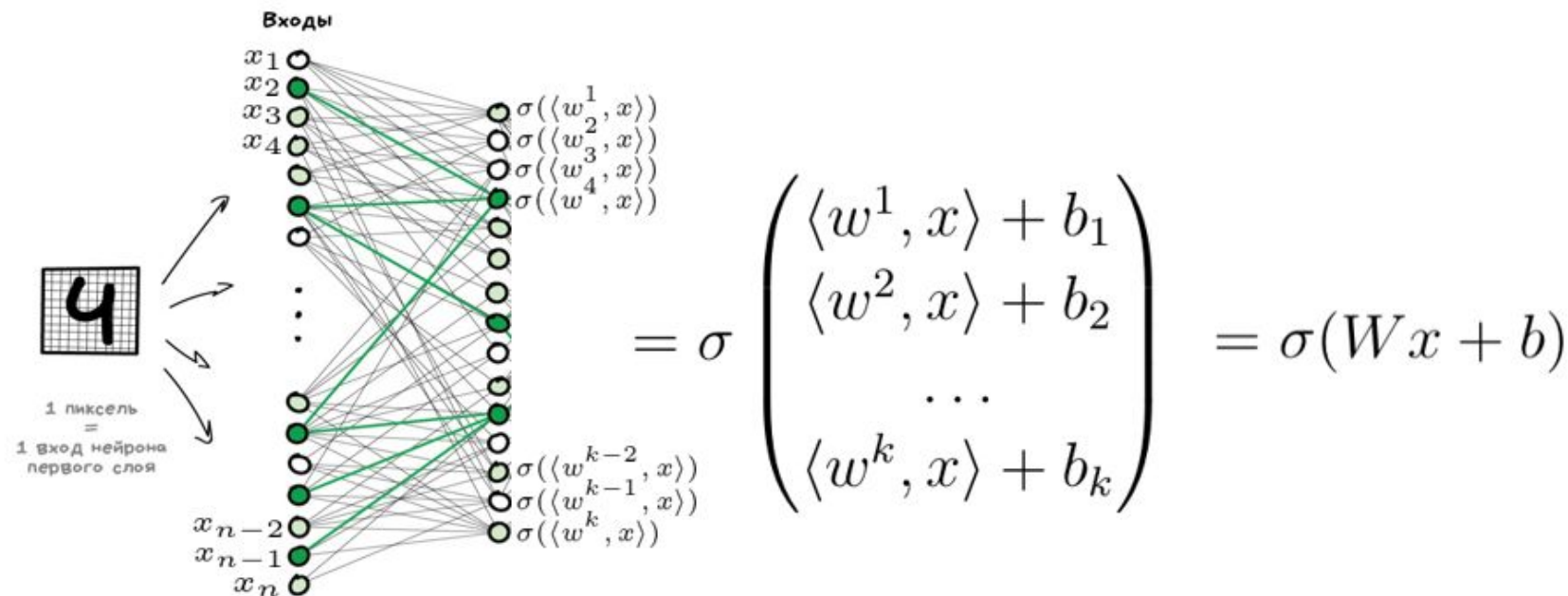
- Многослойный перцептрон — простейшая архитектура нейронной сети
- Каждый слой нейронов связан со всем нейронами с предыдущего слоя
- Десять выходных нейронов соответствуют классам (пример)



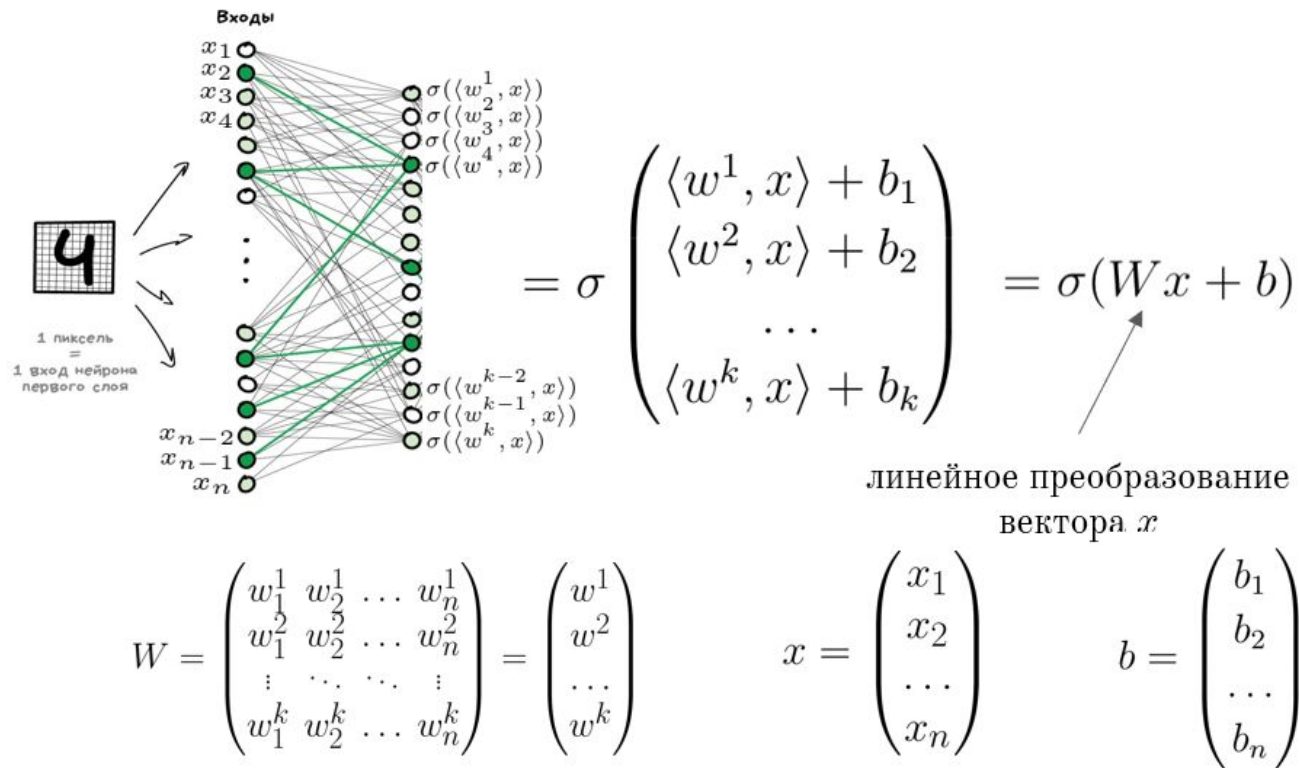
Многослойный перцептрон



Преобразование вектора в перцептроне

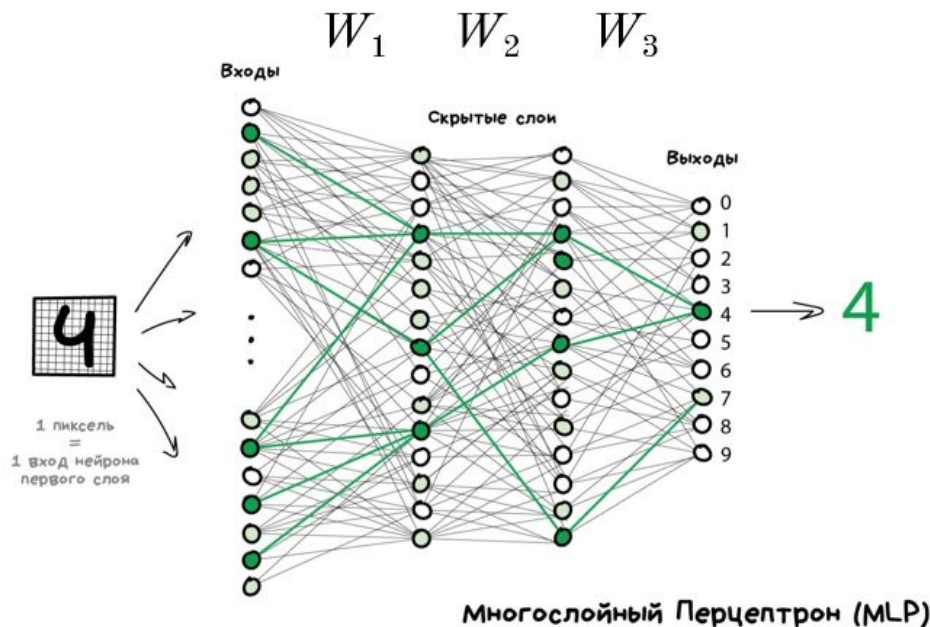


Преобразование вектора в перцептроне



Параметры нейронной сети

- $(W_1, W_2, W_3, b_1, b_2, b_3)$ — совокупность параметров нейронной сети



Трехслойная FCNN

Трехслойная нейронная сеть. Она состоит из трех основных слоев: входного, скрытого и выходного. Каждый слой включает несколько нейронов, которые соединены между собой с помощью весов (обозначены как W).

Основные элементы сети:

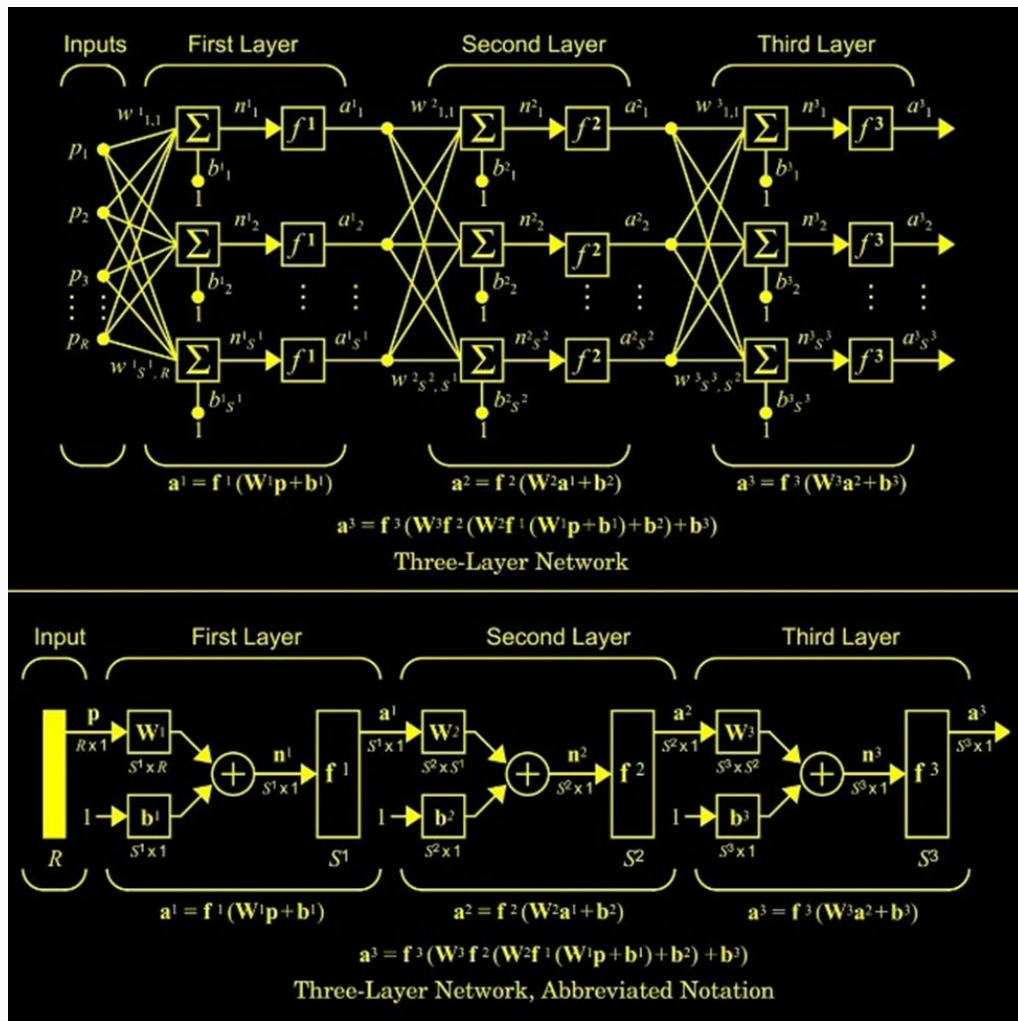
- Входной слой: Принимает входные данные P и передает их на первый скрытый слой.
- Скрытый слой: Выполняет обработку данных с использованием весов $W1$ и смещений $b1$. Результат обработки передается на второй скрытый слой.
- Выходной слой: Генерирует итоговый результат, используя веса $W3$ и смещения $b3$.

Формулы:

1. Для первого слоя: $a^1 = f^1(W^1p + b^1)$.
2. Для второго слоя: $a^2 = f^2(W^2a^1 + b^2)$.
3. Для третьего слоя: $a^3 = f^3(W^3a^2 + b^3)$.

Обозначения:

- P — входные данные.
- $W1, W2, W3$ — матрицы весов для каждого слоя.
- b^1, b^2, b^3 — смещения для каждого слоя.
- f^1, f^2, f^3 — функции активации.



*Обучение нейронных сетей

- Процедуры для обучения нейронных сетей
 - Произвести инференс на обучающем наборе
 - Вычислить ошибку между спрогнозированными значениями и истинными значениями на обучающем наборе
 - Определите вклад каждого нейрона в ошибку
 - Изменить веса нейронной сети для минимизации ошибки
- Вклад в размер ошибки вычисляется с помощью обратного распространения (Backpropagation)
- Минимизация ошибки достигается благодаря градиентному спуску (Gradient Descent)

*Метод градиентного спуска

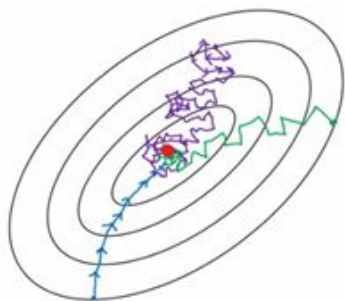
- Метод градиентного спуска минимизирует ошибку нейронной сети
 - На каждой итерации ошибка сети рассчитывается на основе обучающих данных
 - Затем модифицируются веса для уменьшения ошибки
- Метод градиентного спуска останавливается когда:
 - Ошибка достаточно мала
 - Максимальное количество итераций превышено

Стохастический градиентный спуск

- Выбираем батч примеров из обучающей выборки
- Вычисляем производную функции потерь по всем весам нейросети
- Обновляем веса в направлении антиградиента

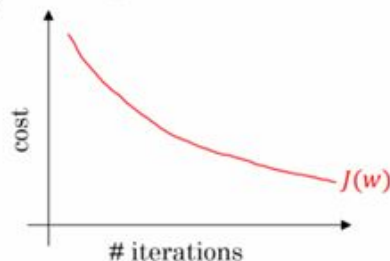
*Батчи

- Функцию потерь считаем по одному примеру, но потом их складываем в одну
- Обучать на всем датасете – долго. Каждый раз берем небольшую порцию данных
- Но обучение становится хаотичнее

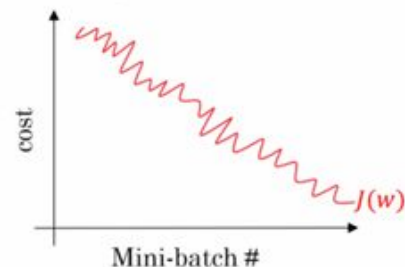


— Batch gradient descent
— Mini-batch gradient Descent
— Stochastic gradient descent

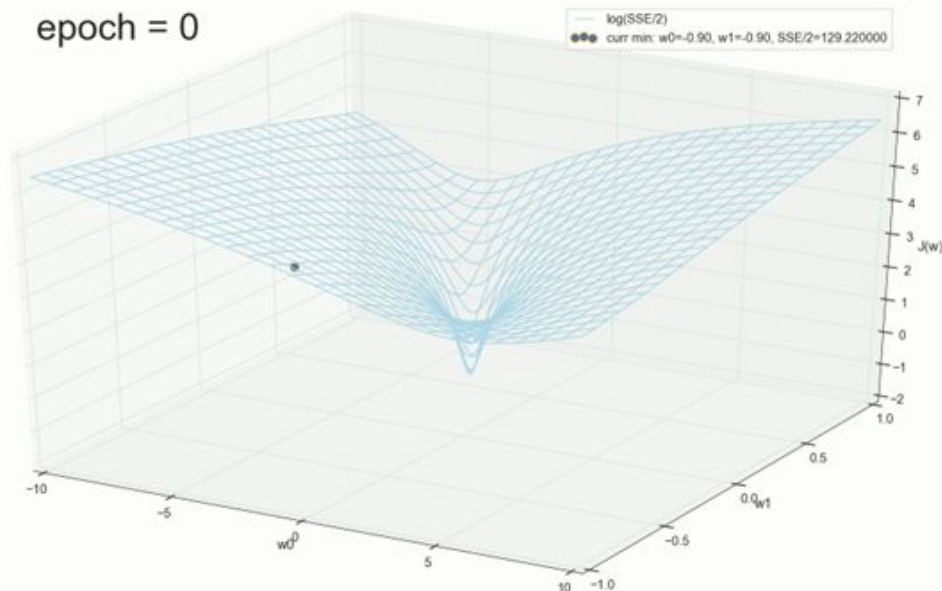
Batch gradient descent



Mini-batch gradient descent

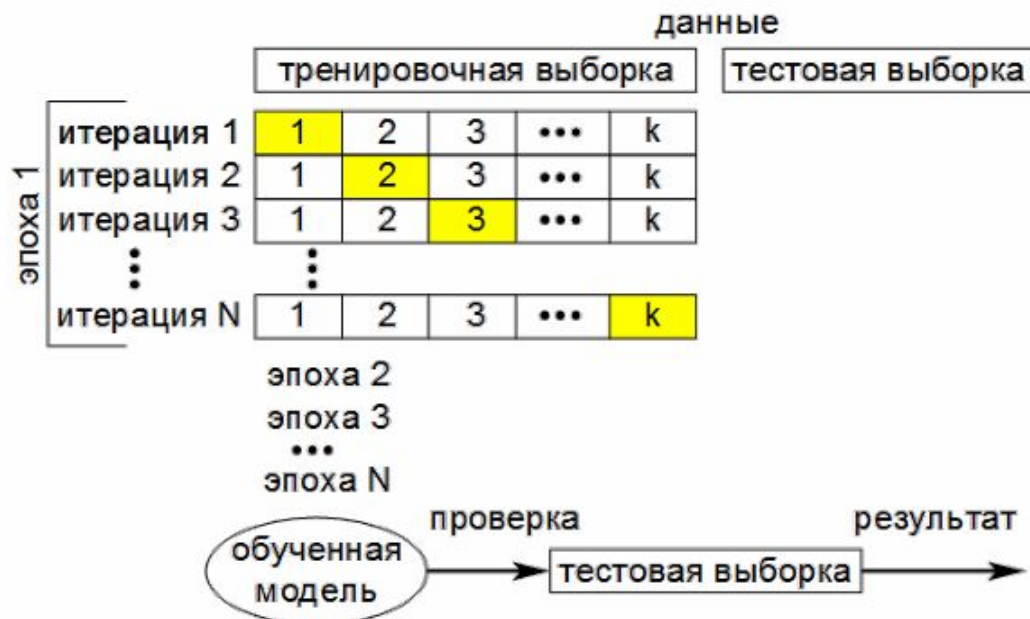


epoch = 0



*Итерации и эпохи

- Итерации – один шаг обучения
- Эпоха – обход всех экземпляров набора данных



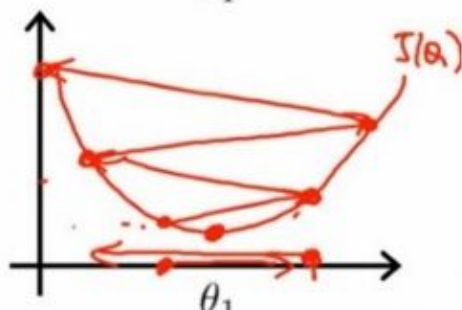
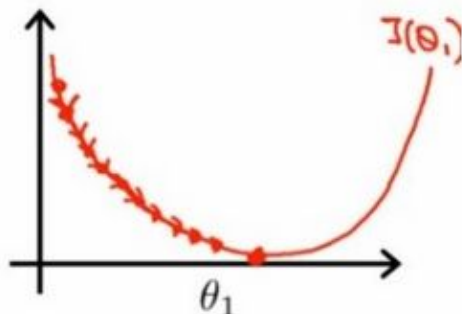
*Скорость обучения

- Параметры (веса) сети меняются при обучении
- Гиперпараметры – нет. Управляем обучением

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

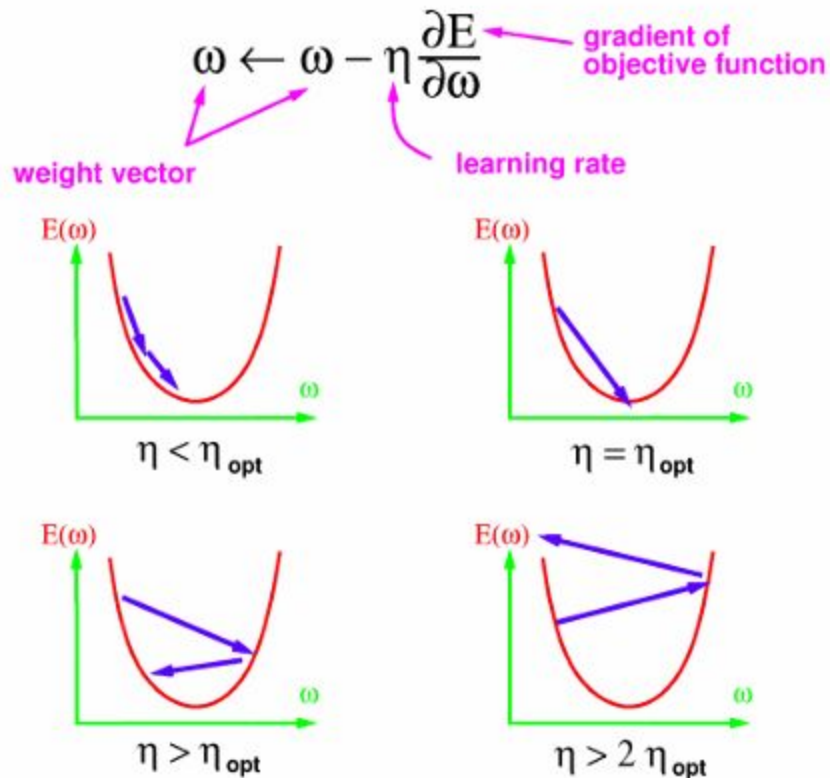
Если α слишком маленькая, то градиентный спуск может быть слишком медленным.

Если α слишком большая, то градиентный спуск может не попасть в точку минимума. Кроме того, сходимость может быть не достигнута.



Немного теории

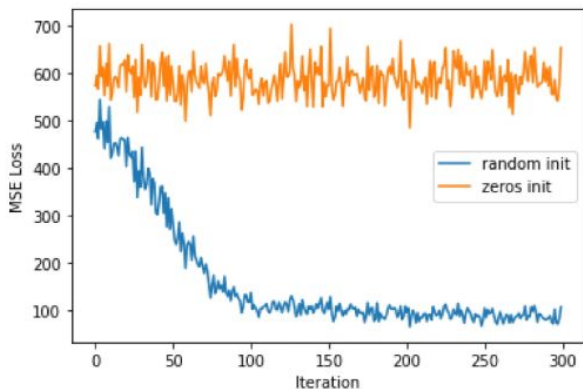
Градиентный спуск в одном измерении



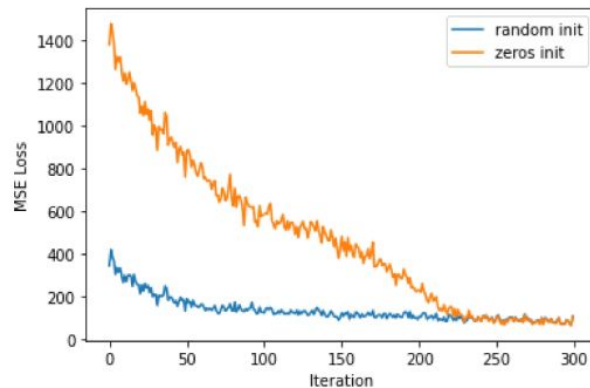
Инициализация весов. Регуляризация нейросетей. Dropout и Batchnorm

Наивный подход №0: инициализация нулем/константой

Казалось бы, инициализация параметров слоя нулями — это достаточно просто и лаконично. Но инициализация нулём (как и любой другой константой) ведёт к катастрофе! Вот пример того, что может получиться:



Стоит, впрочем, отметить, что из-за численных ошибок значения параметров могут всё-таки сдвинуться с мёртвой точки, и тогда нейросеть что-нибудь выучит:



Инициализация Ксавьера (Xavier / Glorot): Масштабирует начальные веса в зависимости от количества входных нейронов (размерности слоя n). Формула дисперсии для нормального распределения: $Var(W) = \frac{1}{n}$. Подходит для функций активации вроде сигмоида или гиперболического тангенса.

Инициализация Хе (He): Учитывает специфику работы с функцией активации ReLU. Дисперсия весов рассчитывается по формуле: $Var(W) = \frac{2}{n}$

Регуляризация нейронных сетей

Смысл термина **регуляризация** (англ. *regularization*) гораздо шире привычного вам

прибавления L_1 - или L_2 -нормы вектора весов к функции потерь. Фактически он объединяет большое количество техник для борьбы с переобучением и для получения более подходящего решения с точки зрения эксперта.

Каждая из них позволяет навязать модели определённые свойства, пусть даже и ценой некоторого снижения качества предсказания на обучающей выборке.

Например, уже знакомая читателю L_1 - или L_2 -регуляризация в задаче линейной регрессии (регуляризация Тихонова) позволяет исключить наименее значимые признаки (для линейной модели) или же получить устойчивое (хоть и смещённое) решение в случае мультиколлинеарных признаков.

В нейронных сетях техники регуляризации можно разделить на три обширные группы:

- связанные с изменением функции потерь;
- связанные с изменением структуры сети;
- связанные с изменением данных.

Рассмотрим каждую из них подробнее.

Регуляризация через функцию потерь

Изменение функции потерь — классический способ получить решение, удовлетворяющее определённым условиям. В глубинном обучении часто используется техника Weight Decay, очень близкая к регуляризации Тихонова. Она представляет собой аналогичный штраф за высокие значения весов нейронной сети

с коэффициентом регуляризации λ :

$$L_{\text{with regularization}} = L_{\text{original}} + \lambda ||\mathbf{W}||_2$$

Данная техника регуляризации была совмещена с методом градиентной оптимизации Adam, в результате чего был получен метод AdamW (описанный в параграфе [параграфе](#) про методы оптимизации).

Также достаточно часто в качестве регуляризационного члена встречается энтропия распределения, предсказанного нейронной сетью.

Представьте, что вы рекомендуете пользователю товары по истории его взаимодействия с сервисом, семплируя товары для показа в соответствии с распределением предсказанной релевантности. Вам может быть важно, чтобы рекомендации не были фиксированными (менялись при обновлении страницы), ведь это повысит вероятность того, что пользователь найдёт что-то интересное, а вы узнаете о нём что-нибудь новое.

В такой ситуации при обучении модели вы можете потребовать, чтобы распределение предсказаний не сходилась к вырожденному, и в качестве дополнительной штрафной функции может выступать энтропия этого распределения. Энтропия дифференцируема, как и сами предсказанные величины, и может быть использована в качестве регуляризационного члена. Для задачи классификации он будет выглядеть следующим образом:

$$\hat{\mathbf{p}} = f(x; \theta),$$

$$L_{\text{with regularization}} = L_{\text{original}} - \lambda \sum_k \hat{p}_k \log \hat{p}_k,$$

где λ — коэффициент регуляризации, $\hat{\mathbf{p}}$ — предсказанные вероятности.

Тем самым эксперт привносит своё знание непосредственно в процесс обучения модели в подходящей математической форме: «предсказания должны быть разнообразными» $\rightarrow$ «распределение не должно быть вырожденным» $\rightarrow$ «энтропия не должна быть слишком низкой».

Регуляризация через ограничение структуры модели

Внесение подходящих преобразований в структуру сети также может быть хорошим способом добиться желаемых результатов. Огромное влияние на развитие нейронных сетей оказали техники [dropout \(2014\)](#) и [batch normalization \(2015\)](#), позволившие сделать нейронные сети более устойчивыми к переобучению и многократно ускорить их сходимость соответственно.

Dropout

Обратимся к простым полносвязным (FC/Dense) сетям из нескольких слоёв.

Каждый из слоёв порождает новое признаковое описание x^k объекта x^{in} , который пришёл на вход:

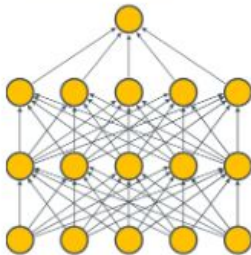
$$x^k = f_k(x^{k-1}).$$

Но как можно гарантировать, что модель будет эффективно использовать все доступные параметры, а не переобучится под использование лишь небольшого их подмножества, поделив для себя внутреннее представление на сигнал и шум?

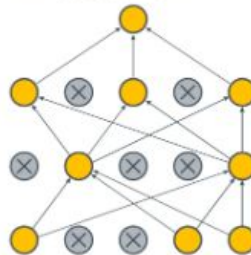
$$x_{\text{overfitted}}^k = [\text{signal}, \text{noise}]$$

Для этого можно было бы случайным образом «выключать» доступ к некоторым координатам внутренних представлений на этапе обучения. Тогда при выключении «полезных» координат произойдёт резкое изменение предсказаний модели, что приведёт к увеличению ошибки, а полученные градиенты этой ошибки укажут, как её исправить с использованием (и изменением) других координат. Сравнение тока информации по исходной модели и по модели с «выключенными» координатами внутренних представлений можно проиллюстрировать с помощью классической картинки:

Standard Neural Net



After applying dropout



Обратите внимание: «выключать» можно как оригинальные признаки, так и признаки, возникающие на любом другом уровне представления объектов. С точки

зрения $(k + 1)$ -го слоя нейронной сети данные приходят откуда-то извне: при

$k = 0$ — из реального мира, а при $k > 1$ — с предыдущих слоёв.

Технически это осуществляется следующим образом: некоторые координаты внутреннего представления домножаются на ноль. То есть добавляется ещё одно преобразование, которое представляет собой домножение выхода предыдущего слоя на маску из нулей и единиц.

$$x^{k+1} = \frac{1}{1-p} x^k \odot \text{mask},$$

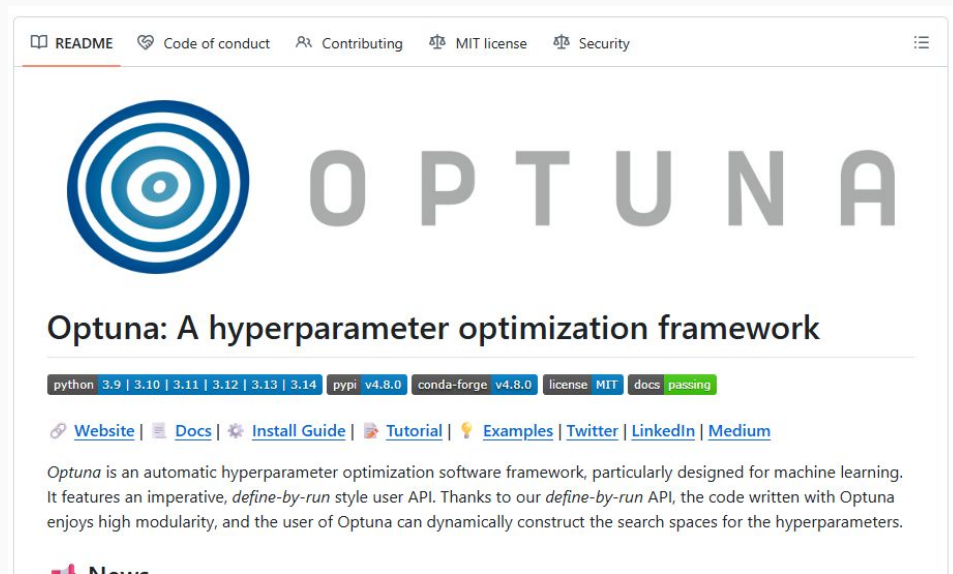
$$\text{mask}_i \sim \text{Bernoulli}(1-p).$$

где $(1-p)$ (вероятность обнуления координаты) — это гиперпараметр слоя.

Отметим, что во многих фреймворках для глубокого обучения в качестве параметра слоя указывается именно вероятность обнуления, а не выживания.

Тюнинг гиперпараметров - Optuna

<https://github.com/optuna/optuna>



	AutoSampler	RandomSampler	TPESampler	GPSampler	CmaEsSampler	NSGAIISampler
Float parameters	✓	✓	✓	✓	✓	▲
Integer parameters	✓	✓	✓	✓	✓	▲
Categorical parameters	✓	✓	✓	✓	▲	✓
Pruning	▲	✓	✓	▲	▲	✗ (▲ for single-objective)
Multivariate optimization	✓	▲	✓	✓	✓	▲
Conditional search space	✓	✓	✓	▲	▲	▲
Multi-objective optimization	✓	✓	✓	✓	✗	✓ (▲ for single-objective)
Batch optimization	✓	✓	✓	▲	✓	✓
Distributed optimization	✓	✓	✓	▲	✓	✓
Constrained optimization	✓	✗	✓	✓	✗	✓
Time complexity (per trial) (*)	N/A	$O(d)$	$O(dn \log n)$	$O(n^3)$	$O(d^3)$	$O(mp^2)$ (***)
Recommended budgets (#trials) (**)	as many as one likes	as many as one likes	100–1000	–500	1000–10000	100–10000

NSGAIISampler	NSGAIISampler	GridSampler	QMCsampler	BoTorchSampler	BruteForceSampler
▲	▲	✓	✓	✓	✓ (✗ for infinite domain)
▲	▲	✓	✓	▲	✓
✓	✓	✓	▲	▲	✓
✗ (▲ for single-objective)	✗ (▲ for single-objective)	✓	✓	▲	✓
▲	▲	▲	▲	✓	▲
▲	▲	▲	▲	▲	✓
✓ (▲ for single-objective)	✓ (▲ for single-objective)	✓	✓	✓	✓
✓	✓	✓	✓	✓	✓
✓	✓	✓	✓	✓	✓
✓	✓	✗	✗	✓	✗
$O(mp^2)$ (***)	$O(mp^2)$ (***)	$O(dn)$	$O(dn)$	$O(n^3)$	$O(d)$
100-10000	100-10000	number of combinations	as many as one likes	10-100	number of combinations

Сложность по времени

1. $O(1)$ - константное время

- Время не меняется независимо от размера входа.
- Пример: доступ к элементу массива по индексу.

2. $O(\log n)$ - логарифмическое время

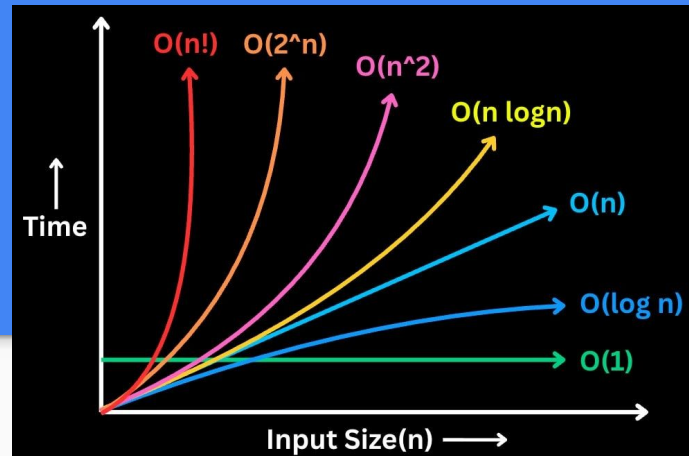
- Растет медленно по мере увеличения входа. Обычно в алгоритмах, которые на каждом шаге делят задачу пополам.
- Пример: бинарный поиск в отсортированном массиве.

3. $O(n)$ - линейное время

- Растет линейно вместе с размером входа.
- Пример: поиск элемента в массиве перебором.

4. $O(n \log n)$ - линеарифмическое время

- Растет чуть быстрее, чем линейное. Для каждого элемента входа делается логарифмическое число операций.
- Пример: сортировка массива quick sort или merge sort.



5. $O(n^2)$ - квадратичное время

- Растет пропорционально квадрату размера входа.
- Пример: bubble sort, который сравнивает и при необходимости меняет местами каждую пару элементов.

6. $O(2^n)$ - экспоненциальное время

- Время удваивается при добавлении каждого элемента во вход. Для больших n становится непрактично.
- Пример: генерация всех подмножеств множества.

7. $O(n!)$ - факториальное время

- Время пропорционально факториалу размера входа.
- Пример: генерация всех перестановок множества.